

Trabalho Final Sistema de Recomendação

PEDRO AUGUSTO MOTA BARRETO DA ROCHA
PEDRO VITOR DE SOUZA DANTAS

O trabalho a seguir, mostra experimentos da disciplina Sistema de Recomendação, ministrada pelo professor Filipe Braidá. O link para o repositório com os códigos utilizados é <https://github.com/PedroVSD/sistema-de-recomendacao>

ACM Reference Format:

Pedro Augusto Mota Barreto da Rocha, Pedro Vitor de Souza Dantas. 2018. Trabalho Final Sistema de Recomendação. *ACM Trans. Graph.* 37, 4, Article 111 (August 2018), 4 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introdução

Pesquisas relacionadas ao desenvolvimento de sistemas de recomendação têm se tornado cada vez mais frequentes, dada a necessidade crescente de auxiliar os usuários a lidar com a vasta quantidade de informações e a encontrar itens de seu real interesse. Nesse contexto, o presente trabalho tem como objetivo analisar e comparar diferentes algoritmos de filtragem colaborativa (abordagens baseadas em memória e modelo) utilizando a base de dados pública do MovieLens 100k. Além da implementação padrão, a pesquisa avalia como a mudança nos valores dos hiperparâmetros trazem um impacto direto na capacidade de generalização e no desempenho final das recomendações.

2 Experimentos

O trabalho foi desenvolvido utilizando três algoritmos de recomendação. Dois deles pertencem à família *memory-based*, enquanto o terceiro pertence à família *model-based*.

Na abordagem *memory-based*, foi utilizado o algoritmo *K-Nearest Neighbors (KNN)* tanto na estratégia *user-based* quanto na estratégia *item-based*. Em ambos os casos, diferentes configurações de hiperparâmetros foram avaliadas, buscando melhorar o desempenho dos modelos.

Já na abordagem *model-based*, foi utilizado o algoritmo *SVD by Funk*. Foram realizados diversos testes com diferentes conjuntos de parâmetros, com o objetivo de minimizar o erro de treinamento e reduzir problemas de *overfitting*.

2.1 Base de Dados

Foi utilizado o conjunto de dados do MovieLens 100k para o desenvolvimento do trabalho, que foi coletado pelo Projeto de Pesquisa GroupLens, na Universidade de Minnesota. A escolha dessa base de dados justifica-se por ser um benchmark amplamente consolidado

na literatura acadêmica para a avaliação de Sistemas de Recomendação de Filtragem Colaborativa.

Esta base consiste em 100.000 avaliações (em uma escala de 1 a 5 estrelas) de 943 usuários em 1682 filmes, de modo que cada usuário tenha avaliado pelo menos 20 filmes.

Tendo em vista o objetivo do trabalho, as informações de timestamp foram desconsideradas, focando-se estritamente no Usuário, Item e Nota. Para a execução dos algoritmos baseados em memória (KNN), os dados brutos foram transformados em uma matriz bidimensional (Usuário $\times$ Filme). Os espaços correspondentes a filmes não assistidos por um determinado usuário foram inicialmente preenchidos com zero para viabilizar os cálculos de métricas de distância e correlação. Para a abordagem baseada em modelo (SVD), o DataFrame foi mantido em seu formato original, durante o treinamento do modelo utilizando Gradiente Descendente.[1][2]

Para garantir robustez metodológica nos experimentos executados, foi utilizada a técnica *hold-out* para a divisão da base de 100.000 avaliações. Os subconjuntos gerados foram:

- **Treino (80%):** utilizado exclusivamente para a construção do conhecimento dos algoritmos, incluindo o cálculo de vizinhanças no KNN e o ajuste de pesos e vieses no SVD.
- **Validação (10%):** utilizado como um ambiente isolado de testes iterativos para a otimização dos hiperparâmetros.
- **Teste (10%):** conjunto de dados isolado, executado apenas ao final das etapas para extrair o Erro Quadrático Médio (RMSE) final, garantindo que os resultados reportados reflitam a precisão real do modelo diante de dados totalmente inéditos.

2.2 Algoritmos

Nesta seção são apresentados os algoritmos utilizados no desenvolvimento do sistema de recomendação.

2.2.1 Memory-based.

Conforme mencionado anteriormente, o algoritmo baseado em memória utilizado foi o *K-Nearest Neighbors (KNN)* em duas abordagens distintas: *user-based* e *item-based*. A seguir são apresentadas as formulações matemáticas utilizadas para cada abordagem.

A predição de notas no modelo *user-based* é dada por:

$$\tilde{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_i(u)} w_{uv} \cdot (r_{vi} - \bar{r}_v)}{\sum_{v \in N_i(u)} |w_{uv}|} \quad (1)$$

Já no modelo *item-based*, a predição é definida por:

$$\tilde{r}_{ui} = \bar{r}_i + \frac{\sum_{j \in N_u(i)} w_{ij} \cdot (r_{uj} - \bar{r}_j)}{\sum_{j \in N_u(i)} |w_{ij}|} \quad (2)$$

Em ambas as equações, $\tilde{r}_{ui}$ representa a nota predita para o usuário u no item i . Os termos w_{uv} e w_{ij} representam, respectivamente, as similaridades entre usuários e entre itens. Além disso, $\bar{r}_u$ e $\bar{r}_i$ correspondem às médias das avaliações do usuário e do item.

Na heurística do KNN foi utilizado a similaridade de Pearson.

Author's Contact Information: Pedro Augusto Mota Barreto da Rocha
Pedro Vitor de Souza Dantas.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7368/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

Similaridade de Pearson:

$$w_{uv} = \frac{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)^2} \cdot \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \bar{r}_v)^2}} \quad (3)$$

2.2.2 Model-based.

No model-based utilizamos o algoritmo SVD by funk[1][2]. Que consiste em descobrir os K fatores latentes associados ao usuário e ao item e a partir disso modelar padrões implícitos nas avaliações e consequentemente previsões mais precisas.

Esses K fatores latentes são as variáveis que o modelo, durante o seu treinamento aprende para que possa ter uma descrição mais adequada de determinado dado.

Além disso o modelo também conta com o bias, para usuário e item. O bias tenta representar a tendencia individual do usuário ou item, melhorando a modelagem e os resultados do modelo. esse hiper parâmetro atua para tentar melhorar a previsão do modelo.

A predição das avaliações é dada por:

$$\tilde{r}_{ui} = \mu + b_u + b_i + p_u^T q_i \quad (4)$$

Em que μ representa a média global das avaliações, b_u e b_i correspondem ao *bias* do usuário e do item, respectivamente, enquanto p_u e q_i representam os vetores de fatores latentes do usuário e do item.

O processo de treinamento consiste em minimizar a seguinte função já com a regularização aplicada:

$$\min_{b, p, q} \sum_{(u, i) \in R} (r_{ui} - \tilde{r}_{ui})^2 + \lambda (b_u^2 + b_i^2 + \|p_u\|^2 + \|q_i\|^2) \quad (5)$$

Em que λ é o parâmetro de regularização utilizado para reduzir problemas de *overfitting*.

Os parâmetros do modelo são atualizados de forma iterativa por meio do método de descida do gradiente estocástico (*Stochastic Gradient Descent* – SGD) [1]:

- $b_u \leftarrow b_u + \gamma(e_{ui} - \lambda b_u)$
- $b_i \leftarrow b_i + \gamma(e_{ui} - \lambda b_i)$
- $p_u \leftarrow p_u + \gamma(e_{ui} q_i - \lambda p_u)$
- $q_i \leftarrow q_i + \gamma(e_{ui} p_u - \lambda q_i)$

Em que γ representa a taxa de aprendizado e e_{ui} corresponde ao erro da predição para o usuário u no item i .

2.3 Metodologia

A metodologia do trabalho se deu da seguinte forma. O conjunto de dados MovieLens 100K foi dividido em três partes utilizando a técnica *hold-out*, onde dividimos 80% para treino, 10% validação e 10% para testes. Também foi garantindo uma padronização na divisão desses subconjuntos para uma reprodutabilidade do experimento.

Dada essa separação, os algoritmos eram treinados e então comparados. Buscando sempre o menor erro no conjunto de treino e claro, tentando evitar o *overfitting* e *underfitting*.

No KNN antes do treino do algoritmo os dados foram normalizados via *z-score*:

$$r_{ui} = \frac{x_{ui} - \mu_u}{\sigma_u} \quad (6)$$

- r_{ui} : Nota do usuário normalizada

- x_{ui} : O valor individual que você quer analisar.
- μ : A média da população.
- σ : O desvio padrão da população.

E logo após a predição esses valores eram desnormalizados. E também utilizamos o clipping para ter um controle das notas para seguir o padrão dos dados utilizados, *notas de 1 até 5*. Utilizamos também a abordagem *Top-N* onde os cinco primeiros itens são as recomendações feitas pelo modelo.

$$\tilde{r}_{ui} = \mu_u + (\tilde{x}_{ui} \cdot \sigma_u) \quad (7)$$

2.3.1 Hiperparametrização.

Na etapa de hiperparametrização, foram definidas faixas de valores para os parâmetros selecionados nas abordagens baseadas em memória e modelo, visando mensurar o impacto dessas mudanças no desempenho das recomendações através do RMSE (Raiz do Erro Quadrático Médio), o que permitiu estimar com precisão a melhor combinação de hiperparâmetros para os modelos.

$$RMSE = \sqrt{\frac{1}{|T|} \sum_{(u, i) \in T} (r_{ui} - \tilde{r}_{ui})^2} \quad (8)$$

Tabela 1. Hiperparâmetros testados para a abordagem Memory-Based (KNN), baseados em [3].

Hiperparâmetro	Valores Testados
Vizinhos (K)	[5, 10, 20, 60, 80, 100]

Tabela 2. Hiperparâmetros testados para a abordagem Model-Based (Funk SVD), baseados em [1].

Hiperparâmetro	Valores Testados
Fatores Latentes (K)	[10, 20, 50, 100, 200]

2.3.2 Model based Hiperparametrização.

Overfitting: Conforme o número de variáveis latentes K aumenta de 10 para 200, observa-se uma redução drástica e contínua no erro de treino, onde o RMSE cai de 0,8798 para 0,5599. Esse comportamento indica que o modelo ganha uma capacidade muito alta de memorizar a base histórica, um sobreajuste. No entanto, os erros de validação e teste não acompanham essa melhora; eles se estabilizam e começam a diminuir pouca coisa para valores elevados de K (atingindo o pior RMSE de teste de 0,9489 em $K = 200$). Isso caracteriza um cenário clássico de *overfitting*, mostrando que fatores latentes em excesso capturam ruídos em vez de padrões generalizáveis.

Melhor Escolha (K): O valor de $K = 50$ demonstrou ser a escolha ideal para o modelo final. Ele oferece o menor erro de generalização no conjunto de teste (RMSE de 0,9359) e um erro de validação controlado (0,9293), alcançando o melhor compromisso entre viés e variância antes que o *overfitting* comece a prejudicar o sistema.

Tabela 3. Resultados experimentais do Funk SVD variando o número de fatores latentes (K). Os valores abaixo utilizam $\gamma = 0.005$ e $\lambda = 0.02$.[1]

Fatores (K)	Tempo de Execução (s)	RMSE Treino	RMSE Validação	RMSE Teste
10	115,23	0,8798	0,9291	0,9410
20	112,33	0,8527	0,9313	0,9416
50	124,12	0,7770	0,9293	0,9359
100	117,79	0,6890	0,9329	0,9422
200	123,65	0,5599	0,9333	0,9489

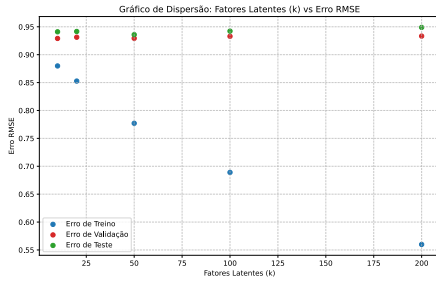


Fig. 1. Gráfico de dispersão: Fatores latentes vs Erro RMSE

2.3.3 Memory based Hiperparametrização.

2.3.3.1 User based. Os resultados obtidos na hiperparametrização do algoritmo baseado em memória *User based* (Table 4) demonstram que a seleção de um número muito pequeno de vizinhos ($k = 5$ e $k = 10$) torna o modelo altamente suscetível a ruídos. Consequentemente, essas configurações apresentaram os maiores índices de erro (RMSE de Teste de 1,0548 e 1,0195, respectivamente). Conforme a vizinhança é expandida, observa-se uma queda acentuada na métrica de erro (Fig. 2). No entanto, a partir de $k = 60$, a curva entra em um estágio de estabilidade, onde o acréscimo de novos vizinhos passa a ter um impacto pouco expressivo, alterando os resultados dos conjuntos de Validação e Teste somente nas últimas casas decimais.

Portanto, a partir desses dados e análise do gráfico (Fig. 3), conclui-se que o valor ótimo para a vizinhança encontra-se na faixa entre $k = 20$ e $k = 80$. Essa janela representa o ponto de equilíbrio ideal do modelo: é ampla o suficiente para garantir o ganho de informação e filtrar o ruído individual dos usuários, mas restrita o bastante para evitar que a recomendação perca a sua personalização ao incluir na vizinhança usuários com gostos muito divergentes do perfil alvo.

Tabela 4. Resultados da Avaliação do Algoritmo KNN (*User based*) por Número de Vizinhos

Vizinhos (k)	Tempo de Execução (s)	RMSE Treino	RMSE Validação	RMSE Teste
5	7,34	0,9942	1,0366	1,0548
10	6,17	0,9624	1,0000	1,0195
20	6,21	0,9489	0,9845	1,0024
60	6,14	0,9449	0,9771	0,9956
80	6,24	0,9451	0,9768	0,9952
100	6,05	0,9453	0,9767	0,9952

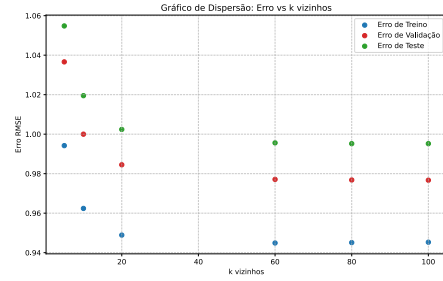


Fig. 2. Gráfico de dispersão: k vizinhos vs Erro RMSE

2.3.3.2 Item based.

Os resultados obtidos na hiperparametrização do algoritmo baseado em memória *Item based* (Table 5) demonstram que a seleção de um número muito pequeno de vizinhos ($K = 5$ e $k = 10$) torna o modelo suscetível a ruídos, como foi observado no *User based*. Consequentemente, essas configurações apresentaram os maiores índices de erro (RMSE de Teste de 1,0290 e 0,9996, respectivamente). Conforme o número de vizinhos aumenta (maior valor de K), percebe-se uma boa queda na métrica de erro (Fig. 3). No entanto, a partir de $K = 60$, a curva fica bem instável, onde o acréscimo de novos vizinhos passa a ter um impacto pouco expressivo, alterando os resultados dos conjuntos de Validação e Teste somente nas últimas casas decimais.

Portanto, a partir desses dados e análise do gráfico (Fig. 3), conclui-se que o valor ótimo para a vizinhança encontra-se na faixa entre $k = 20$ e $k = 80$. Essa janela representa o ponto de equilíbrio ideal do modelo: é ampla o suficiente para garantir o ganho de informação e filtrar o ruído nas correlações entre os itens, mas também restrita de modo a não incluir muitos itens na vizinhança.

Tabela 5. Resultados da Avaliação do Algoritmo KNN (*Item based*) por Número de Vizinhos

Vizinhos (k)	Tempo de Execução (s)	RMSE Treino	RMSE Validação	RMSE Teste
5	21,63	0,9665	1,0191	1,0290
10	21,75	0,9358	0,9829	0,9996
20	23,49	0,9231	0,9702	0,9839
60	22,07	0,9194	0,9642	0,9776
80	22,29	0,9194	0,9641	0,9774
100	22,66	0,9194	0,9641	0,9775

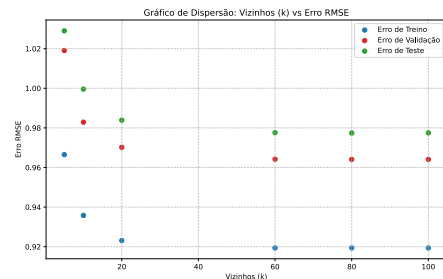


Fig. 3. Gráfico de dispersão: K vizinhos no modelo item-based vs Erro RMSE

Tabela 6. Comparativo entre o histórico de notas máximas do Usuário 1 e as recomendações geradas pelos algoritmos.

Pos.	Histórico (Nota 5)	User-Based (KNN)	Item-Based (KNN)	Funk SVD
1	202 – Groundhog Day	408 – Close Shave, A	251 – Shall We Dance?	408 – Close Shave, A
2	64 – Shawshank Redemption	695 – Kicking and Screaming	313 – Titanic	246 – Chasing Amy
3	177 – Good, Bad and Ugly	814 – Great Day in Harlem	357 – Cuckoo’s Nest	268 – Chasing Amy
4	114 – Wallace & Gromit	817 – Frisk	408 – Close Shave, A	646 – Once Upon a Time...
5	228 – Star Trek: Khan	851 – Two or Three Things...	483 – Casablanca	963 – Some Folks Call...

2.4 Uma análise do Top-N (Usuário 1)

A Tabela 6 apresenta o mapeamento das preferências históricas do Usuário 1 em contraste com as cinco principais recomendações geradas pelos algoritmos avaliados. A partir do histórico de avaliações (notas máximas), observa-se que o usuário apresenta um perfil predominantemente eclético, transitando entre clássicos do cinema, animações e produções independentes.

Os resultados revelam assinaturas comportamentais distintas para cada abordagem:

- **Abordagem baseada em Itens (Item-Based):** Demonstrou maior proximidade ao histórico. A presença de *Close Shave, A* (Posição 4) mostra a forte correlação estatística com a obra *Wallace & Gromit*, pertencente ao mesmo estúdio de animação e presente no histórico do usuário. Além disso, a inclusão de dramas cult como *Casablanca* valida a capacidade do algoritmo em identificar a similaridade entre itens de alta avaliação global.
- **Abordagem baseada em Usuários (User-Based):** Direcionou as recomendações para um circuito mais alternativo e de nicho ("o lado cinéfilo"). Ao agrupar vizinhos com gostos correlatos, o algoritmo sugeriu títulos como *Two or Three Things...*, aproximando-se do interesse do usuário por cinema europeu e produções independentes dos anos 1990.
- **Abordagem baseada em Modelo (Funk SVD):** Através da decomposição em fatores latentes, o modelo capturou nuances estruturais que ultrapassam a similaridade direta de categorias. O maior destaque reside na recomendação de *Once Upon a Time in the West*. O SVD identificou com sucesso o viés latente associado à obra *The Good, The Bad and The Ugly*, uma vez que ambos compartilham o mesmo diretor, estilo conceitual (*Spaghetti Western*) e assinaturas técnicas.

Diante do exposto, destaca-se a convergência dos três algoritmos na recomendação de *Close Shave, A*. Esse consenso matemático evidencia que o item possui alta similaridade tanto no espaço de características dos produtos quanto no comportamento da vizinhança de usuários, consolidando-se como a recomendação de maior confiança do sistema.

3 Conclusão

O trabalho apresentou uma análise comparativa entre abordagens de filtragem colaborativa baseadas em memória (KNN *User-based* e *Item-based*) e modelo (SVD). Os resultados dos experimentos indicaram a importância da etapa de hiperparametrização para a qualidade das recomendações geradas, sendo de grande importância avaliar a métrica de erro RMSE nos conjuntos de treino, validação e teste,

para obter o melhor modelo e evitar problemas como *overfitting* e *underfitting* como auxílio para a tomada de decisão.

Nos modelos baseados em memória, constatou-se que o tamanho da vizinhança afeta diretamente a precisão do sistema. A utilização de um número muito restrito de vizinhos (como $k = 5$ e $k = 10$) torna o modelo altamente suscetível a ruídos e avaliações isoladas, resultando em altos índices de erro. Em contrapartida, a expansão excessiva desse valor dilui a qualidade da recomendação, pois passa a incluir usuários ou itens com características muito divergentes do perfil alvo, estagnando a redução do erro. Portanto, observou-se que a faixa entre $k = 20$ e $k = 80$ representa o ponto de equilíbrio e generalização ideal, balanceando o ganho de informação com a manutenção da personalização.

Para o cenário da abordagem baseada em modelo, foi avaliado como a variação do número de fatores latentes (k factors) influencia a qualidade das recomendações e de que forma um aumento excessivo nesse parâmetro pode levar o algoritmo a um cenário de *overfitting*. Os testes evidenciaram que uma expansão acentuada dessa variável (até k factors = 200) reduz o erro RMSE de treino devido à alta memorização da base histórica, porém os dados de validação e teste não acompanharam essa melhoria, indicando que o excesso de fatores passa a capturar ruídos além dos padrões úteis.

Em termos de desempenho geral, pode-se concluir que a abordagem baseada em modelo demonstrou uma capacidade preditiva superior quando devidamente parametrizada (com $k_factors = 50$), lidando de forma mais robusta e eficiente com a esparsidade da matriz de avaliações em comparação com os algoritmos baseados em memória. Para trabalhos futuros, é sugerido explorar Sistemas de Recomendação Híbridos, incorporando técnicas baseadas em conteúdo (*Content-Based*) para mitigar as limitações puramente colaborativas. Além disso, recomenda-se a incorporação de métricas adicionais que capturem outras características do modelo, permitindo uma análise mais abrangente para os resultados obtidos.

Referências

- [1] KOREN, Yehuda; RENDLE, Steffen; BELL, Robert. Advances in Collaborative Filtering. In: RICCI, Francesco; ROKACH, Lior; SHAPIRA, Bracha (Ed.). **Recommender Systems Handbook**. 3. ed. New York: Springer, 2022. cap. 3, p. 91-142.
- [2] FUNK, Simon. **Netflix update: try this at home**. 2006. Disponível em: <http://sifter.org/~simon/journal/20061211.html>. Acesso em: 24 jun. 2026.
- [3] RICCI, Francesco; ROKACH, Lior; SHAPIRA, Bracha (Ed.). Trust Your Neighbors: A Comprehensive Survey of Neighborhood-Based Methods for Recommender Systems. In: **Recommender Systems Handbook**. 3. ed. New York: Springer, 2022. cap. 2.
- [4] RICCI, Francesco; ROKACH, Lior; SHAPIRA, Bracha (Ed.). **Recommender Systems Handbook**. 3. ed. New York: Springer, 2022.